Lab 8: Heuristic Search Algorithms

AI Module — ATU Galway
Topic: Hill Climbing, Best-First Search, Beam Search, A*, and Enriched Evaluation Functions Libraries:* NetworkX, matplotlib, heapq

---

Learning Objectives

By the end of this lab, you will be able to:

1. **Implement from scratch** the following heuristic search algorithms:
   - Hill Climbing (Steepest Ascent)
   - Greedy Best-First Search
   - Beam Search
   - A* Search
2. **Explain** how heuristic evaluation functions guide informed search.
3. **Demonstrate** the limitations of hill climbing (foothills, plateaux).
4. **Trace** the A* OPEN and CLOSED lists, including the computation of $f(n) = g(n) + h(n)$.
5. **Compare** uninformed and informed search on the same problem.
6. **Use** different cost functions $f^*(n)$ to show how A* adapts to richer problem models.

Setup

In [ ]:
```
import networkx as nx
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import heapq
from collections import deque
import numpy as np

%matplotlib inline
plt.rcParams['figure.figsize'] = (10, 7)
plt.rcParams['figure.dpi'] = 100
```

Part 1: Building a Weighted Graph with Heuristic Values

1.1 The Ireland Map

In Lab 7, our maze had unweighted edges — every step cost the same. In the real world, edges have different **costs** (e.g., distances between cities). Heuristic search algorithms exploit **additional knowledge** about the problem — specifically, an estimate of how far each node is from the goal — to search more efficiently.

We'll use a map of Ireland as our search problem. The goal is to find the best route from **Galway** to **Waterford**.

Each city (node) has:

- **Edges** with **actual distances** (in km) to neighbouring cities.
- A **heuristic value** $h(n)$: the estimated straight-line ("crow-flies") distance to Waterford.

1.2 Building the Graph

In [ ]:
```
# Build the Ireland map as a weighted graph
ireland = nx.Graph()

# Add edges with distances (km)
ireland.add_edge('Galway', 'Limerick', weight=105)
ireland.add_edge('Galway', 'Belfast', weight=306)
# TODO: add the remaining edges & distances (see this cell's output)


# Heuristic: straight-line distance to Waterford (km)
heuristic = {
    'Galway': 200, 'Limerick': 170, 'Belfast': 270,
    'Cork': 120, 'Dublin': 130, 'Waterford': 0
}

# Store heuristic as node attribute
for city, h in heuristic.items():
    ireland.nodes[city]['h'] = h

print("Cities:", list(ireland.nodes()))
print("\nConnections and distances:")
for u, v, data in ireland.edges(data=True):
    print(f"  {u} <-> {v}: {data['weight']} km")
print("\nHeuristic values (crow-flies distance to Waterford):")
```

```
for city, h in sorted(heuristic.items(), key=lambda x: x[1]):
    print(f"  {city}: h = {h} km")
```

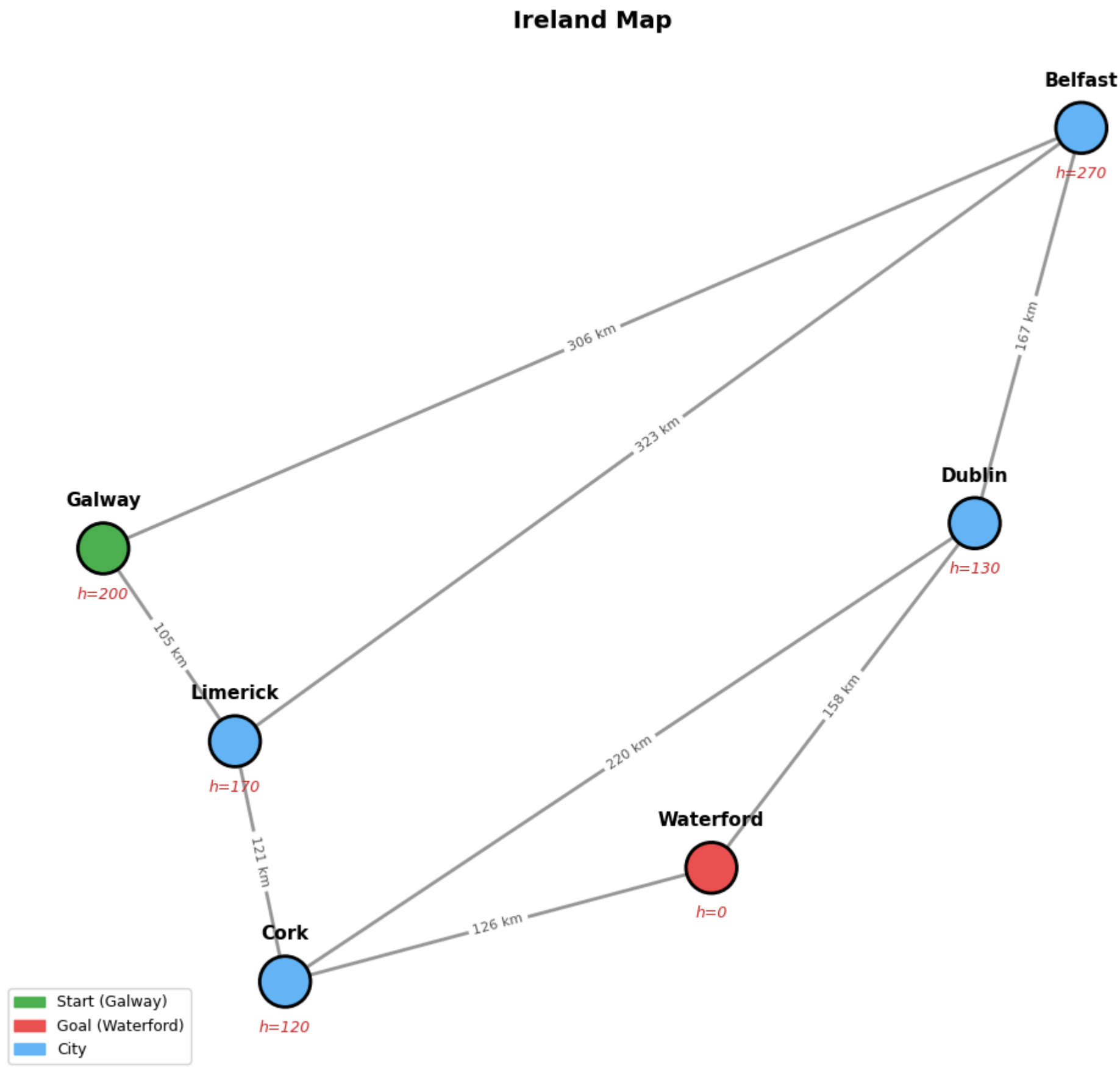
```
Cities: ['Galway', 'Limerick', 'Belfast', 'Cork', 'Dublin', 'Waterford']

Connections and distances:
  Galway <-> Limerick: 105 km
  Galway <-> Belfast: 306 km
  Limerick <-> Belfast: 323 km
  Limerick <-> Cork: 121 km
  Belfast <-> Dublin: 167 km
  Cork <-> Dublin: 220 km
  Cork <-> Waterford: 126 km
  Dublin <-> Waterford: 158 km

Heuristic values (crow-flies distance to Waterford):
  Waterford: h = 0 km
  Cork: h = 120 km
  Dublin: h = 130 km
  Limerick: h = 170 km
  Galway: h = 200 km
  Belfast: h = 270 km
```

1.3 Visualising the Ireland Map

In [ ]:

```
# Geographic-ish positions for Irish cities
ireland_pos = {
    'Galway':    (-9.05, 53.27),
    'Limerick':  (-8.63, 52.66),
    'Cork':      (-8.47, 51.90),
    'Waterford': (-7.11, 52.26),
    'Dublin':    (-6.27, 53.35),
    'Belfast':   (-5.93, 54.60),
}

def draw_ireland(graph, pos, h, title='Ireland Map', highlight_path=None,
                 highlight_cost=None, extra_nodes=None, extra_h=None):
    """Draw the Ireland map with distances and heuristic values."""
    fig, ax = plt.subplots(figsize=(10, 9))

    all_h = dict(h)
    if extra_h:
        all_h.update(extra_h)

    # Node colours
    node_colors = []
    for n in graph.nodes():
        if n == 'Galway':
            node_colors.append('#4CAF50')
        elif n == 'Waterford':
            node_colors.append('#EF5350')
        else:
            node_colors.append('#64B5F6')

    # Draw edges with distance labels
    nx.draw_networkx_edges(graph, pos, ax=ax, edge_color='#999999', width=2)
    edge_labels = {(u, v): f"{d['weight']} km" for u, v, d in graph.edges(data=True)}
    nx.draw_networkx_edge_labels(graph, pos, edge_labels=edge_labels, ax=ax,
                                 font_size=8, font_color='#555555')

    # Highlight path if provided
    if highlight_path:
        path_edges = list(zip(highlight_path[:-1], highlight_path[1:]))
        nx.draw_networkx_edges(graph, pos, edgelist=path_edges,
                               edge_color='#E91E63', width=2, ax=ax)

    # Draw nodes
    nx.draw_networkx_nodes(graph, pos, ax=ax, node_color=node_colors,
                           node_size=900, edgecolors='black', linewidths=2)

    # Labels: city name and h(n) value
    for node, (x, y) in pos.items():
        ax.text(x, y + 0.12, node, ha='center', va='bottom',
                fontsize=11, fontweight='bold')
        if node in all_h:
            ax.text(x, y - 0.12, f'h={all_h[node]}', ha='center', va='top',
                    fontsize=9, color='#C62828', fontstyle='italic')

    # Legend
    legend_items = [
        mpatches.Patch(color='#4CAF50', label='Start (Galway)'),
        mpatches.Patch(color='#EF5350', label='Goal (Waterford)'),
        mpatches.Patch(color='#64B5F6', label='City'),
    ]
    if highlight_path:
        cost_str = f' (cost: {highlight_cost} km)' if highlight_cost else ''
        legend_items.append(mpatches.Patch(color='#E91E63',
                           label=f'Path: {" → ".join(highlight_path)}{cost_str}'))
    ax.legend(handles=legend_items, loc='lower left', fontsize=9)

    ax.set_title(title, fontsize=14, fontweight='bold')
    ax.axis('off')
```

```
    plt.tight_layout()
    plt.show()

draw_ireland(ireland, ireland_pos, heuristic)
```



1.4 Admissibility and Consistency

Before we use the heuristic, we should check two important properties.

Admissibility: A heuristic $h(n)$ is **admissible** if it never overestimates the true cost to reach the goal: $h(n) \leq h^*(n)$ for all nodes n, where $h^*(n)$ is the actual cheapest cost from n to the goal. An admissible heuristic guarantees that A* finds an optimal solution.

Consistency (monotonicity): A heuristic is **consistent** if for every node n and every successor n': $h(n) \leq c(n, n') + h(n')$, where $c(n, n')$ is the edge cost. This is the triangle inequality. Consistency implies admissibility, and it ensures that $f(n)$ values along any path are non-decreasing.

> **Note:** Consistency is sometimes confused with *heuristic dominance* — which is a different concept. Dominance compares two heuristics: h_1 dominates h_2 if $h_1(n) \geq h_2(n)$ for all n (meaning h_1 is more informed). Consistency is a property of a single heuristic.

In [ ]:
```
# Check admissibility: h(n) <= actual shortest distance to Waterford
print("Admissibility Check")
print("=" * 60)
# TODO: Use the appropriate NetworkX function to compute into "shortest_to_goal" the shortest
#       path lengths from Waterford to all other reachable cities using Dijkstra's algorithm.

for city in sorted(ireland.nodes()):
    actual = shortest_to_goal[city]
    h = heuristic[city]
    admissible = h <= actual
    print(f"  {city:>10}: h={h:>3}, actual shortest={actual:>3}, "
          f"h <= actual? {admissible} {'✓' if admissible else '✗ NOT ADMISSIBLE'}")

# Check consistency: h(n) <= c(n,n') + h(n') for all edges
print(f"\nConsistency Check (Triangle Inequality)")
print("=" * 60)
all_consistent = True
for u, v, data in ireland.edges(data=True):
    c = data['weight']
```

```
    for n, np_ in [(u, v), (v, u)]:
        satisfied = heuristic[n] <= c + heuristic[np_]
        if not satisfied:
            all_consistent = False
        print(f"  h({n})={heuristic[n]} <= c({n},{np_})={c} + h({np_})={heuristic[np_]} "
              f"= {c + heuristic[np_]}? {satisfied} {'✓' if satisfied else '✗'}")

print(f"\nHeuristic is {'consistent (and therefore admissible)' if all_consistent else 'NOT consistent'}.")
```

```
Admissibility Check
============================================================
     Belfast: h=270, actual shortest=325, h <= actual? True ✓
        Cork: h=120, actual shortest=126, h <= actual? True ✓
      Dublin: h=130, actual shortest=158, h <= actual? True ✓
      Galway: h=200, actual shortest=352, h <= actual? True ✓
    Limerick: h=170, actual shortest=247, h <= actual? True ✓
   Waterford: h=  0, actual shortest=  0, h <= actual? True ✓

Consistency Check (Triangle Inequality)
============================================================
  h(Galway)=200 <= c(Galway,Limerick)=105 + h(Limerick)=170 = 275? True ✓
  h(Limerick)=170 <= c(Limerick,Galway)=105 + h(Galway)=200 = 305? True ✓
  h(Galway)=200 <= c(Galway,Belfast)=306 + h(Belfast)=270 = 576? True ✓
  h(Belfast)=270 <= c(Belfast,Galway)=306 + h(Galway)=200 = 506? True ✓
  h(Limerick)=170 <= c(Limerick,Belfast)=323 + h(Belfast)=270 = 593? True ✓
  h(Belfast)=270 <= c(Belfast,Limerick)=323 + h(Limerick)=170 = 493? True ✓
  h(Limerick)=170 <= c(Limerick,Cork)=121 + h(Cork)=120 = 241? True ✓
  h(Cork)=120 <= c(Cork,Limerick)=121 + h(Limerick)=170 = 291? True ✓
  h(Belfast)=270 <= c(Belfast,Dublin)=167 + h(Dublin)=130 = 297? True ✓
  h(Dublin)=130 <= c(Dublin,Belfast)=167 + h(Belfast)=270 = 437? True ✓
  h(Cork)=120 <= c(Cork,Dublin)=220 + h(Dublin)=130 = 350? True ✓
  h(Dublin)=130 <= c(Dublin,Cork)=220 + h(Cork)=120 = 340? True ✓
  h(Cork)=120 <= c(Cork,Waterford)=126 + h(Waterford)=0 = 126? True ✓
  h(Waterford)=0 <= c(Waterford,Cork)=126 + h(Cork)=120 = 246? True ✓
  h(Dublin)=130 <= c(Dublin,Waterford)=158 + h(Waterford)=0 = 158? True ✓
  h(Waterford)=0 <= c(Waterford,Dublin)=158 + h(Dublin)=130 = 288? True ✓

Heuristic is consistent (and therefore admissible).
```

Part 2: Hill Climbing (Steepest Ascent)

2.1 How Hill Climbing Works

Hill climbing is a **local** search algorithm. At each step, it selects the neighbour with the **best (lowest) heuristic value** — the one that *appears* to be closest to the goal. It uses only $h(n)$ (not path cost) and makes **irrevocable** decisions: once it moves to a node, it never backtracks.

Because it only considers local information, hill climbing can be **sidetracked** by:

- **Foothills (local maxima):** A node that looks better than its neighbours but isn't on the optimal path.
- **Plateaux:** A region where multiple nodes have the same heuristic value, giving the algorithm no useful guidance.
- **Ridges:** A narrow high region where the algorithm oscillates without making progress.

Properties:

- **Complete:** No — can get stuck at local optima.
- **Optimal:** No — greedy local decisions don't guarantee global optimality.
- **Time:** $O(d)$ where d is the depth — only follows one path.
- **Space:** $O(1)$ extra (beyond storing the current path).

2.2 Hill Climbing Implementation

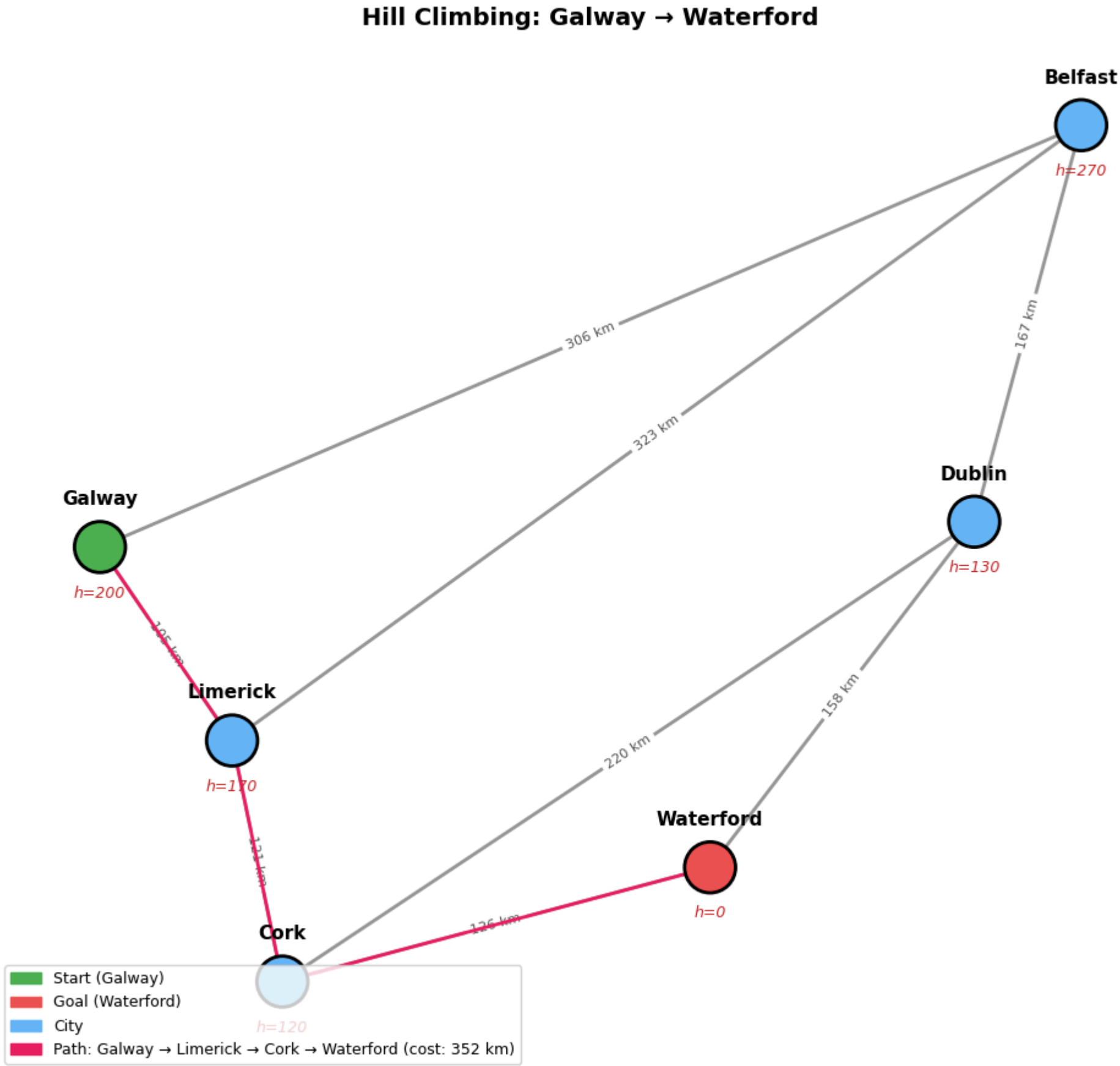
In [ ]:
```
def hill_climbing(graph, h, start, goal):
    """
    Steepest-ascent hill climbing.

    At each step, selects the unvisited neighbour with the lowest h(n).
    Makes irrevocable choices — no backtracking.

    Returns:
        path:       List of nodes from start to goal
        total_cost: Total distance travelled
        found:      Whether the goal was reached
        trace:      Step-by-step trace for analysis
    """
    current = start
    path = [start]
    visited = {start}
    total_cost = 0
    trace = []

    while current != goal:
        neighbors = []
        for neighbor in graph.neighbors(current):
            if neighbor not in visited:
                dist = graph[current][neighbor]['weight']
                neighbors.append((h[neighbor], neighbor, dist))
```

```python
        if not neighbors:
            trace.append({'node': current, 'neighbors': [], 'choice': 'STUCK'})
            return path, total_cost, False, trace

        # Sort by heuristic value (ascending)
        # TODO: Sort neighbors using the proper List sort function


        trace.append({
            'node': current,
            'neighbors': [(n, hv, d) for hv, n, d in neighbors],
            'choice': neighbors[0][1]
        })

        # Choose the best (lowest h) neighbour — irrevocable!
        _, best_node, dist = neighbors[0]
        visited.add(best_node)
        path.append(best_node)
        total_cost += dist
        current = best_node

    trace.append({'node': goal, 'neighbors': [], 'choice': 'GOAL'})
    return path, total_cost, True, trace
```

2.3 Hill Climbing on the Base Map

In [ ]:
```python
# TODO: Run the Hill Climbing search implemented in 2.2


print("Hill Climbing Trace:")
print(f"{'Step':>4}  {'At':>12}  {'Neighbours (name, h, dist)':}  {'Choice':}")
print("-" * 75)
for i, step in enumerate(hc_trace):
    nbrs = ', '.join(f"{n}(h={h},d={d})" for n, h, d in step['neighbors']) if step['neighbors'] else '-'
    print(f"{i+1:>4}  {step['node']:>12}  {nbrs:<40}  {step['choice']}")

print(f"\nPath: {' → '.join(hc_path)}")
print(f"Total distance: {hc_cost} km")

draw_ireland(ireland, ireland_pos, heuristic,
             title='Hill Climbing: Galway → Waterford',
             highlight_path=hc_path, highlight_cost=hc_cost)
```

```
Hill Climbing Trace:
Step            At  Neighbours (name, h, dist)  Choice
---------------------------------------------------------------------------
   1        Galway  Limerick(h=170,d=105), Belfast(h=270,d=306)  Limerick
   2      Limerick  Cork(h=120,d=121), Belfast(h=270,d=323)   Cork
   3          Cork  Waterford(h=0,d=126), Dublin(h=130,d=220)  Waterford
   4     Waterford  -                                          GOAL

Path: Galway → Limerick → Cork → Waterford
Total distance: 352 km
```



Hill climbing finds the optimal path here because our heuristic is well-behaved on this small graph. But what happens when we introduce misleading local optima?

2.4 Foothills: Adding Athlone

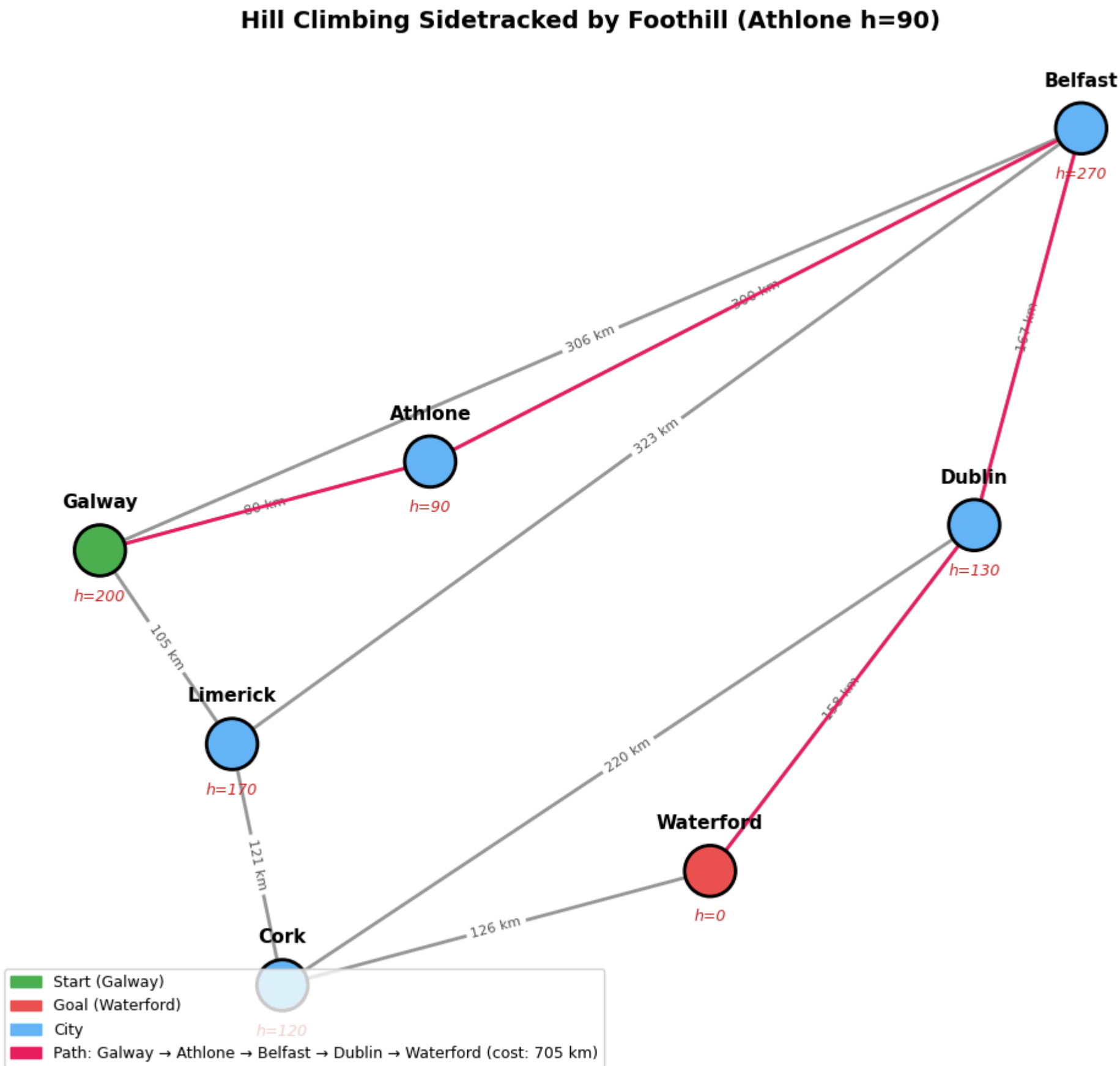
We'll add **Athlone** with a deceptively low heuristic value of $h = 90$ (suggesting it's close to Waterford). However, Athlone only connects to Galway and Belfast — so the only way to Waterford from Athlone is a long detour via Belfast.

This creates a **foothill**: Athlone looks closer to the goal than Limerick ($h = 170$), so hill climbing greedily chooses it, but it leads to a much more expensive path.

In [ ]:
```
# Add Athlone to create a foothill
ireland_fh = ireland.copy()
ireland_fh.add_edge('Galway', 'Athlone', weight=80)
ireland_fh.add_edge('Athlone', 'Belfast', weight=300)

h_fh = dict(heuristic)
h_fh['Athlone'] = 90  # Deceptively low heuristic

# Update positions
pos_fh = dict(ireland_pos)
pos_fh['Athlone'] = (-8.0, 53.55)

# TODO: Run the implemented Hill Climbing search on the map with a foothill


print("Hill Climbing with Athlone (foothill):")
print(f"{'Step':>4}  {'At':>12}  {'Neighbours (name, h, dist)':}  {'Choice':}")
print("-" * 75)
for i, step in enumerate(hc_fh_trace):
    nbrs = ', '.join(f"{n}(h={h},d={d})" for n, h, d in step['neighbors']) if step['neighbors'] else '-'
    print(f"{i+1:>4}  {step['node']:>12}  {nbrs:<40}  {step['choice']}")

print(f"\nPath: {' → '.join(hc_fh_path)}")
print(f"Total distance: {hc_fh_cost} km")
print(f"\nOptimal path was: Galway → Limerick → Cork → Waterford = 352 km")
print(f"Hill climbing was sidetracked: cost {hc_fh_cost} km ({hc_fh_cost - 352} km extra!)")
```

```
draw_ireland(ireland_fh, pos_fh, heuristic, extra_h={'Athlone': 90},
             title=f'Hill Climbing Sidetracked by Foothill (Athlone h=90)',
             highlight_path=hc_fh_path, highlight_cost=hc_fh_cost)
```

```
Hill Climbing with Athlone (foothill):
Step            At  Neighbours (name, h, dist)  Choice
----------------------------------------------------------------------
   1        Galway  Athlone(h=90,d=80), Limerick(h=170,d=105), Belfast(h=270,d=306)  Athlone
   2       Athlone  Belfast(h=270,d=300)                      Belfast
   3       Belfast  Dublin(h=130,d=167), Limerick(h=170,d=323)  Dublin
   4        Dublin  Waterford(h=0,d=158), Cork(h=120,d=220)   Waterford
   5     Waterford  -                                         GOAL

Path: Galway → Athlone → Belfast → Dublin → Waterford
Total distance: 705 km

Optimal path was: Galway → Limerick → Cork → Waterford = 352 km
Hill climbing was sidetracked: cost 705 km (353 km extra!)
```



> **Key lesson:** Hill climbing chose Athlone (h=90) over Limerick (h=170) because Athlone *appeared* closer to the goal. But Athlone's low heuristic was misleading — it was **not admissible** for the path through Athlone ($h = 90$ but the actual shortest distance from Athlone to Waterford via Belfast is $300 + 167 + 158 = 625$ km or $300 + 167 + 158 = 625$ km). The greedy, irrevocable nature of hill climbing means this mistake cannot be corrected.

2.5 Plateaux: Adding Nenagh and Adjusting Athlone

A **plateau** is a region of the search space where multiple nodes share the same heuristic value, giving the algorithm no useful guidance — it cannot distinguish between them.

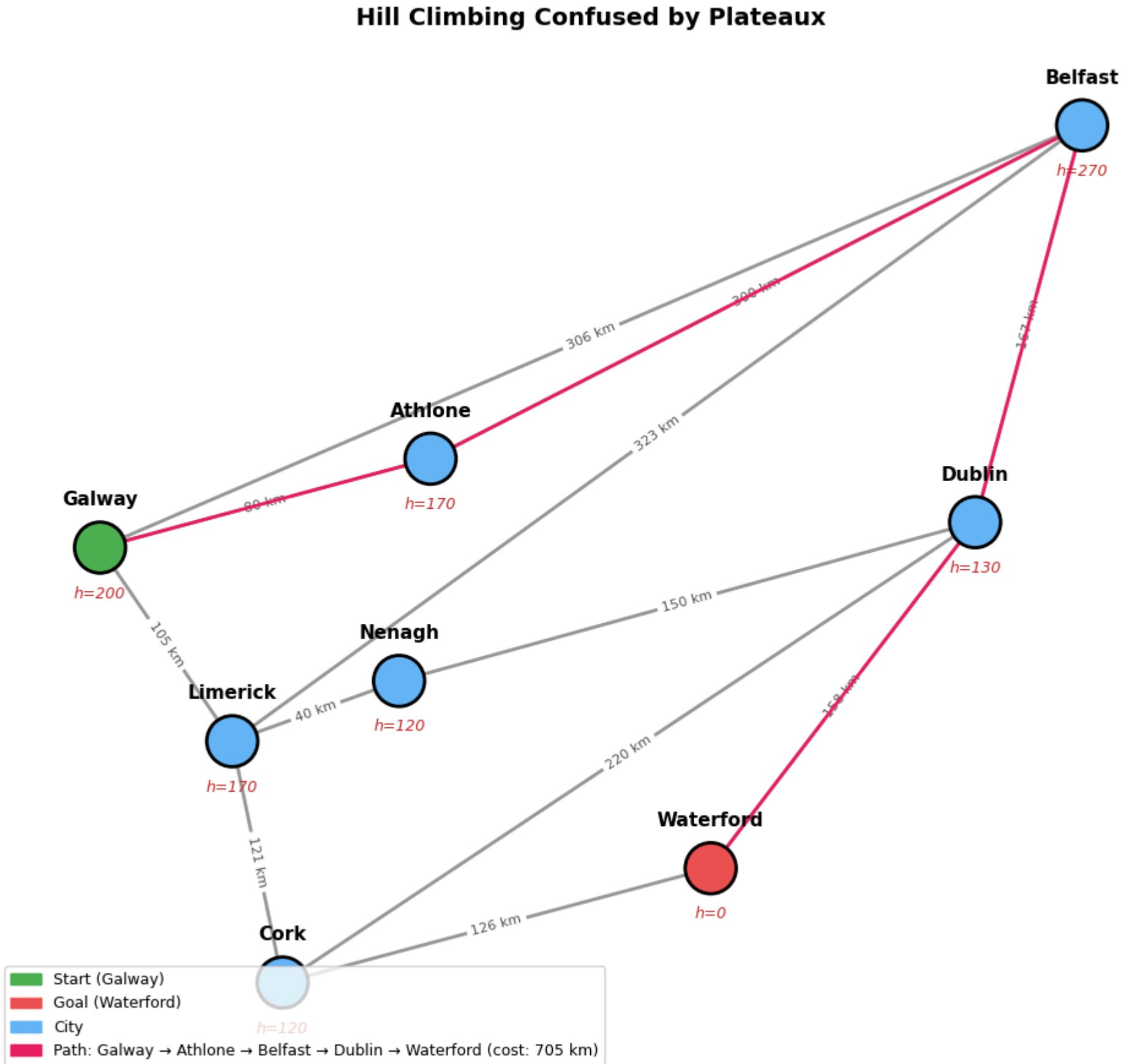
We'll modify the map to create two plateaux:

- Set Athlone's heuristic to **170** (same as Limerick) — a plateau at the first decision point.
- Add **Nenagh** with heuristic **120** (same as Cork), connected to Limerick and Dublin — a second plateau deeper in the search.

In [ ]:
```
# Build the plateau map
ireland_pl = ireland_fh.copy()
```

```python
# Adjust Athlone heuristic to match Limerick (plateau)
h_pl = dict(heuristic)
h_pl['Athlone'] = 170  # same as Limerick!

# TODO: Add Nenagh connected to Limerick and Dublin
## Tip: for example, see how Athlone was added and connected in 2.4
## Weights: Limerick-Nenagh = 40, Nenagh-Dublin = 150
# Add Nenagh with h=120 (same as Cork)


pos_pl = dict(ireland_pos)
pos_pl['Athlone'] = (-8.0, 53.55)
pos_pl['Nenagh'] = (-8.1, 52.85)

# TODO: Run the implemented Hill Climbing search on the map with plateaux


print("Hill Climbing with Plateaux:")
print(f"{'Step':>4}  {'At':>12}  {'Neighbours (name, h, dist)':}  {'Choice':}")
print("-" * 80)
for i, step in enumerate(hc_pl_trace):
    nbrs = ', '.join(f"{n}(h={h},d={d})" for n, h, d in step['neighbors']) if step['neighbors'] else '-'
    # Highlight ties
    if step['neighbors'] and len(step['neighbors']) > 1:
        h_values = [h for n, h, d in step['neighbors']]
        if h_values[0] == h_values[1]:
            nbrs += '  ** TIE **'
    print(f"{i+1:>4}  {step['node']:>12}  {nbrs:<55}  {step['choice']}")

print(f"\nPath: {' → '.join(hc_pl_path)}")
print(f"Total distance: {hc_pl_cost} km")
print(f"Optimal path: Galway → Limerick → Cork → Waterford = 352 km")
print(f"Penalty: {hc_pl_cost - 352} km extra")

draw_ireland(ireland_pl, pos_pl, heuristic,
             extra_h={'Athlone': 170, 'Nenagh': 120},
             title=f'Hill Climbing Confused by Plateaux',
             highlight_path=hc_pl_path, highlight_cost=hc_pl_cost)
```

```
Hill Climbing with Plateaux:
Step            At  Neighbours (name, h, dist)  Choice
--------------------------------------------------------------------------------
   1        Galway  Athlone(h=170,d=80), Limerick(h=170,d=105), Belfast(h=270,d=306)  ** TIE **  Athlone
   2       Athlone  Belfast(h=270,d=300)                                      Belfast
   3       Belfast  Dublin(h=130,d=167), Limerick(h=170,d=323)                Dublin
   4        Dublin  Waterford(h=0,d=158), Cork(h=120,d=220), Nenagh(h=120,d=150)  Waterford
   5     Waterford  -                                                        GOAL

Path: Galway → Athlone → Belfast → Dublin → Waterford
Total distance: 705 km
Optimal path: Galway → Limerick → Cork → Waterford = 352 km
Penalty: 353 km extra
```



> **What happened?** At Galway, the algorithm faces a tie: Athlone and Limerick both have $h = 170$. With no way to distinguish them, the tie-break is arbitrary (here, alphabetical order picks Athlone). This sends the algorithm down the wrong path via Belfast.
>
> Even if the tie had gone to Limerick, a **second plateau** awaits: Cork and Nenagh both have $h = 120$. If that tie goes to Nenagh instead of Cork, the algorithm takes the detour Limerick→Nenagh→Dublin→Waterford (453 km) instead of the optimal Limerick→Cork→Waterford (247 km from Limerick).
>
> **Key lesson:** Plateaux are dangerous because the heuristic provides no guidance in flat regions. The algorithm is essentially making random choices. Unlike foothills (where a single deceptive node causes the problem), plateaux create entire regions of ambiguity.

Part 3: Greedy Best-First Search

3.1 How Greedy Best-First Search Works

Greedy best-first search improves on hill climbing by using a **priority queue** sorted by $h(n)$. Unlike hill climbing, it doesn't make irrevocable decisions — it considers *all* discovered nodes and always expands the one with the lowest $h(n)$, regardless of which path it's on.

This makes it essentially the same as A* with $g(n) = 0$:

$$f(n) = 0 + h(n) = h(n)$$

Properties:

- **Complete:** Yes (on finite graphs with cycle detection).
- **Optimal:** No — ignores path cost $g(n)$.
- **Time:** $O(b^d)$ worst case.
- **Space:** $O(b^d)$ (stores all discovered nodes).

3.2 Greedy Best-First Implementation

In [ ]:
```python
def greedy_best_first(graph, h, start, goal):
    """
    Greedy Best-First Search using a priority queue sorted by h(n).

    Returns:
        path:       Path from start to goal
        total_cost: Actual distance of the path
        expanded:   Number of nodes expanded
        trace:      Step-by-step trace
    """
    visited = set()
    # Priority queue: (h(n), node, path, cost)
    pq = [(h[start], start, [start], 0)]
    expanded = 0
    trace = []

    while pq:
        h_val, node, path, cost = heapq.heappop(pq)

        if node in visited:
            continue
        visited.add(node)
        expanded += 1

        # Record trace
        open_nodes = [(hv, n) for hv, n, _, _ in pq if n not in visited]
        trace.append({
            'visiting': node, 'h': h_val, 'cost_so_far': cost,
            'open': sorted(set(open_nodes)),
            'closed': sorted(visited)
        })

        if node == goal:
            return path, cost, expanded, trace

        for neighbor in graph.neighbors(node):
            if neighbor not in visited:
                dist = graph[node][neighbor]['weight']
                heapq.heappush(pq, (h[neighbor], neighbor,
                                    path + [neighbor], cost + dist))

    return None, 0, expanded, trace
```

In [ ]:
```python
# TODO: Run the implemented Greedy Best First search (3.2) on the original map (1.2)


print("Greedy Best-First Search Trace:")
print(f"{'Step':>4}  {'Visiting':>12}  {'h(n)':>5}  {'Cost':>6}  {'Open Queue':}")
print("-" * 70)
for i, step in enumerate(gbf_trace):
    open_str = ', '.join(f"{n}(h={h})" for h, n in step['open']) if step['open'] else 'empty'
    print(f"{i+1:>4}  {step['visiting']:>12}  {step['h']:>5}  {step['cost_so_far']:>6}  {open_str}")

print(f"\nPath: {' → '.join(gbf_path)}, cost = {gbf_cost} km, nodes expanded = {gbf_exp}")
```

```
Greedy Best-First Search Trace:
Step      Visiting   h(n)    Cost  Open Queue
----------------------------------------------------------------------
   1        Galway    200       0  empty
   2      Limerick    170     105  Belfast(h=270)
   3          Cork    120     226  Belfast(h=270)
   4     Waterford      0     352  Dublin(h=130), Belfast(h=270)

Path: Galway → Limerick → Cork → Waterford, cost = 352 km, nodes expanded = 4
```

Part 4: Beam Search

4.1 How Beam Search Works

Beam search is an optimisation of best-first search that limits memory usage. At each **level** of the search, it keeps only the w **best nodes** (where w is the beam width) and discards the rest. This makes it more space-efficient but risks throwing away nodes on the optimal path.

Properties:

- **Complete:** No — may discard the only path to the goal.
- **Optimal:** No — limited view of the search space.
- **Space:** $O(b \cdot w)$ where w is the beam width.

> **Important:** A proper beam search applies the beam width **per level**, not per node. A per-node limit (keep only w children of each node) is a different, weaker algorithm.

4.2 Beam Search Implementation

In [ ]:
```python
def beam_search(graph, h, start, goal, beam_width=2):
    """
    Beam Search: BFS-style level expansion with a beam width limit.
```

```
    At each level, keeps only the beam_width best nodes (lowest h).

    Returns:
        path:      Path from start to goal
        cost:      Total path cost
        expanded:  Number of nodes expanded
        trace:     Level-by-level trace
    """
    # Each entry: (h, node, path, cost)
    current_level = [(h[start], start, [start], 0)]
    visited = {start}
    expanded = 0
    trace = []

    while current_level:
        trace.append({
            'level_nodes': [(n, hv) for hv, n, _, _ in current_level]
        })

        next_level = []
        for h_val, node, path, cost in current_level:
            expanded += 1

            if node == goal:
                return path, cost, expanded, trace

            for neighbor in graph.neighbors(node):
                if neighbor not in visited:
                    dist = graph[node][neighbor]['weight']
                    next_level.append((h[neighbor], neighbor,
                                       path + [neighbor], cost + dist))

        # TODO: Sort next_level by h(n) using the proper List sort function
        # Keep only beam_width best candidates

        current_level = []
        for item in next_level[:beam_width]:
            _, node, _, _ = item
            if node not in visited:
                visited.add(node)
                current_level.append(item)

    return None, 0, expanded, trace

# TODO: Run the implemented Beam Search on the original map (1.2)
# Beam search with width 2


print(f"Beam Search (w=2): {' → '.join(beam_path)}, cost = {beam_cost} km, "
      f"expanded = {beam_exp}")
print("\nLevel-by-level:")
for i, level in enumerate(beam_trace):
    nodes_str = ', '.join(f"{n}(h={h})" for n, h in level['level_nodes'])
    print(f"  Level {i}: {nodes_str}")

# Compare with width 1 and width 3
for w in [1, 3]:
    # TODO: Run the implemented Beam Search on the original map (1.2) for widths 1 and 3
    # Tip: you don't need beam_trace here, so use a _ instead

    if bp:
        print(f"\nBeam Search (w={w}): {' → '.join(bp)}, cost = {bc} km, expanded = {be}")
    else:
        print(f"\nBeam Search (w={w}): FAILED to find a path")
```

```
Beam Search (w=2): Galway → Belfast → Dublin → Waterford, cost = 631 km, expanded = 6

Level-by-level:
  Level 0: Galway(h=200)
  Level 1: Limerick(h=170), Belfast(h=270)
  Level 2: Cork(h=120), Dublin(h=130)
  Level 3: Waterford(h=0)

Beam Search (w=1): Galway → Limerick → Cork → Waterford, cost = 352 km, expanded = 4

Beam Search (w=3): Galway → Belfast → Dublin → Waterford, cost = 631 km, expanded = 6
```

> **Why does beam width 2 find a worse path than width 1?** This is a subtle but important point. With $w = 1$, beam search keeps only Limerick at level 1 (it has the lowest h), then Cork at level 2, then finds Waterford — the optimal path. With $w = 2$, it keeps *both* Limerick ($h = 170$) and Belfast ($h = 270$) at level 1. When their children are expanded at level 2, the two best nodes are Cork ($h = 120$) and Dublin ($h = 130$) — but Dublin came from the Belfast branch. Dublin then leads to Waterford via a more expensive route. A wider beam doesn't guarantee a better result because beam search is **not optimal** — keeping more candidates can introduce worse paths that happen to look locally promising.

Part 5: A* Search

5.1 How A* Works

A* is the most important algorithm in this lab. It combines:

- $g(n)$**:** the actual cost of the path from the start to node n.
- $h(n)$**:** the heuristic estimate of the cost from n to the goal.

into a single evaluation function:

$$f(n) = g(n) + h(n)$$

A* maintains an **OPEN list** (priority queue sorted by $f(n)$) and a **CLOSED list** (visited nodes). At each step, it expands the node with the lowest $f(n)$.

Properties (with admissible heuristic):

- **Complete:** Yes.
- **Optimal:** Yes — $f(n)$ accounts for both past cost and estimated future cost.
- **Time:** $O(b^d)$.
- **Space:** $O(b^d)$ (stores all discovered nodes in OPEN/CLOSED).

The key insight is that A* never expands a node with $f(n) > f^*(goal)$, where $f^*(goal)$ is the optimal path cost. It prunes the search space using the heuristic while guaranteeing optimality.

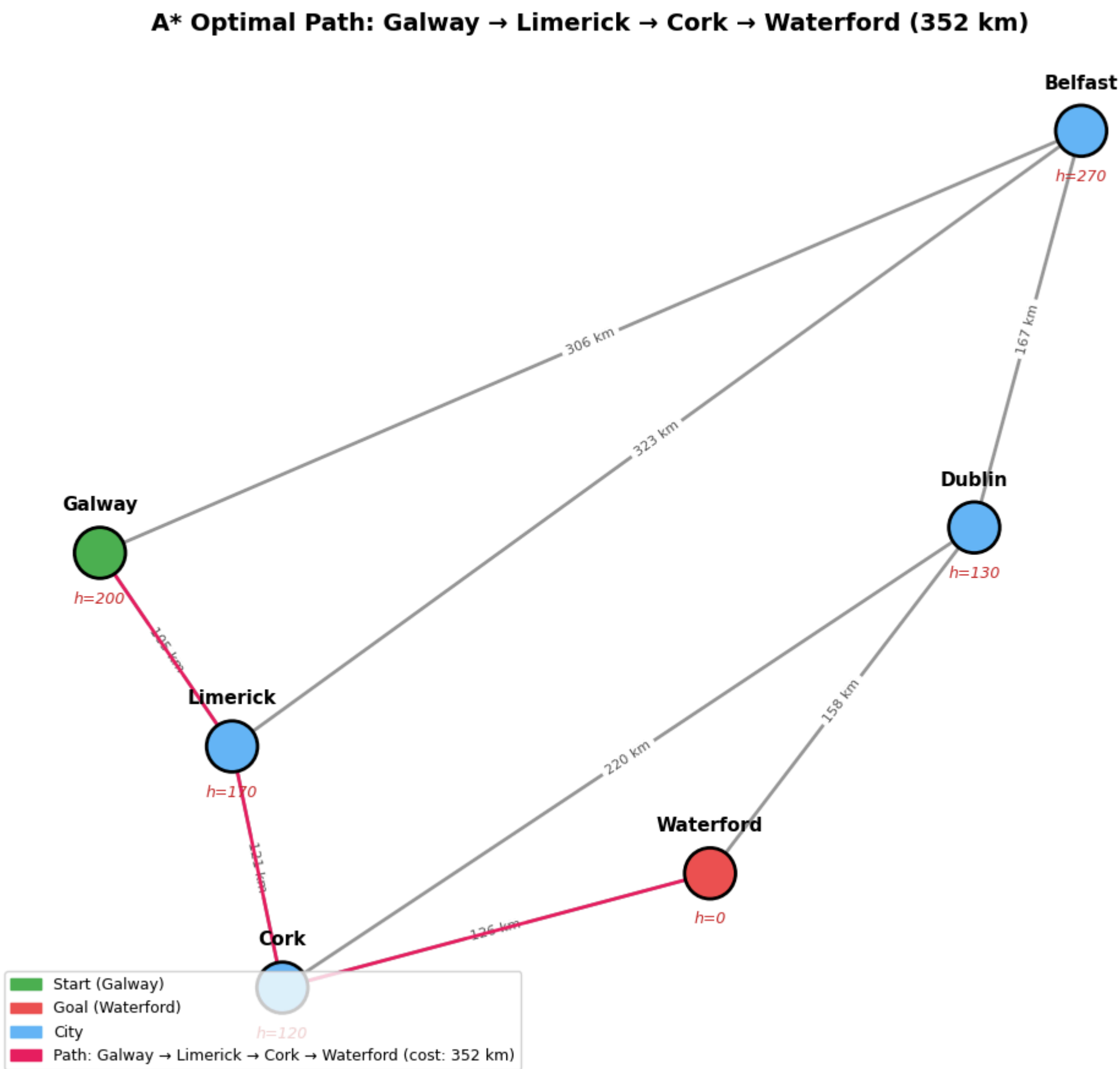
5.2 A* Implementation

In [ ]:
```
def a_star(graph, h, start, goal):
    """
    A* Search with OPEN/CLOSED list tracing.

    Uses f(n) = g(n) + h(n) to select the most promising node.

    Returns:
        path:      Optimal path from start to goal
        cost:      Optimal path cost
        expanded:  Number of nodes expanded
        trace:     Step-by-step trace showing OPEN and CLOSED lists
    """
    visited = set()           # CLOSED list
    g_costs = {start: 0}     # Best known g(n) for each node

    # OPEN list: priority queue of (f(n), g(n), node, path)
    pq = [(h[start], 0, start, [start])]
    expanded = 0
    trace = []

    while pq:
        f_val, g_val, node, path = heapq.heappop(pq)

        if node in visited:
            continue

        visited.add(node)
        expanded += 1

        # Build trace entry
        open_list = []
        for f_, g_, n_, _ in pq:
            if n_ not in visited:
                open_list.append((n_, f_, g_, h[n_]))
        # Deduplicate (keep best f for each node)
        open_dict = {}
        for n_, f_, g_, h_ in open_list:
            if n_ not in open_dict or f_ < open_dict[n_][0]:
                open_dict[n_] = (f_, g_, h_)

        trace.append({
            'visiting': node, 'f': f_val, 'g': g_val, 'h': h[node],
            'open': {n: vals for n, vals in sorted(open_dict.items())},
            'closed': sorted(visited)
        })

        if node == goal:
            return path, g_val, expanded, trace

        # Expand neighbours
        for neighbor in graph.neighbors(node):
            if neighbor not in visited:
                dist = graph[node][neighbor]['weight']
                new_g = g_val + dist

                if neighbor not in g_costs or new_g < g_costs[neighbor]:
                    g_costs[neighbor] = new_g
                    new_f = new_g + h[neighbor]
                    heapq.heappush(pq, (new_f, new_g, neighbor,
                                        path + [neighbor]))

    return None, 0, expanded, trace
```

5.3 A* on the Ireland Map

In [ ]:
```python
# TODO: Run A* implemented in 5.2 on the original map (1.2)


print("A* Search Trace (OPEN and CLOSED lists)")
print("=" * 85)
for i, step in enumerate(astar_trace):
    print(f"\nStep {i+1}: Visit {step['visiting']} "
          f"[f={step['f']}, g={step['g']}, h={step['h']}]")

    if step['open']:
        print(f"  OPEN:   ", end="")
        parts = []
        for n, (f, g, h_) in step['open'].items():
            parts.append(f"{n}(f={f}, g={g}, h={h_})")
        print(', '.join(parts))
    else:
        print(f"  OPEN:   empty")
    print(f"  CLOSED: {step['closed']}")

print(f"\nOptimal path: {' → '.join(astar_path)}")
print(f"Optimal cost: {astar_cost} km")
print(f"Nodes expanded: {astar_exp}")

draw_ireland(ireland, ireland_pos, heuristic,
             title=f'A* Optimal Path: {" → ".join(astar_path)} ({astar_cost} km)',
             highlight_path=astar_path, highlight_cost=astar_cost)
```

```
A* Search Trace (OPEN and CLOSED lists)
=====================================================================================

Step 1: Visit Galway [f=200, g=0, h=200]
  OPEN:   empty
  CLOSED: ['Galway']

Step 2: Visit Limerick [f=275, g=105, h=170]
  OPEN:   Belfast(f=576, g=306, h=270)
  CLOSED: ['Galway', 'Limerick']

Step 3: Visit Cork [f=346, g=226, h=120]
  OPEN:   Belfast(f=576, g=306, h=270)
  CLOSED: ['Cork', 'Galway', 'Limerick']

Step 4: Visit Waterford [f=352, g=352, h=0]
  OPEN:   Belfast(f=576, g=306, h=270), Dublin(f=576, g=446, h=130)
  CLOSED: ['Cork', 'Galway', 'Limerick', 'Waterford']

Optimal path: Galway → Limerick → Cork → Waterford
Optimal cost: 352 km
Nodes expanded: 4
```



> **Note on the trace above:** The OPEN list appears empty at Step 1 (visiting Galway) because our trace records the OPEN list *before* the current node's neighbours are added. In an exam-style trace, you would typically show the OPEN list *after* expansion. The subsequent steps show the correct OPEN contents. This is a cosmetic difference in presentation — the algorithm's behaviour is correct. For example, we could add this before Step 1:

```
Step 0: Visit Galway [f=200, g=0, h=200]
OPEN: ['Galway']
CLOSED: empty
```

5.4 A* with the Athlone Foothill

Unlike hill climbing, A* is not fooled by the Athlone foothill because it considers the *total* cost $f(n) = g(n) + h(n)$, not just $h(n)$.

In [ ]:
```
h_fh_full = dict(heuristic)
h_fh_full['Athlone'] = 90

# TODO: Run A* implemented in 5.2 on the map with a foothill (2.4)
#       using h_fh_full (line above) as heuristic


print("A* with Athlone foothill:")
print("=" * 85)
for i, step in enumerate(astar_fh_trace):
    print(f"\nStep {i+1}: Visit {step['visiting']} "
          f"[f={step['f']}, g={step['g']}, h={step['h']}]")
    if step['open']:
        print(f"  OPEN:   ", end="")
        parts = []
        for n, (f, g, h_) in step['open'].items():
            parts.append(f"{n}(f={f}, g={g}, h={h_})")
        print(', '.join(parts))
    print(f"  CLOSED: {step['closed']}")

print(f"\nA* path: {' → '.join(astar_fh_path)}, cost = {astar_fh_cost} km")
print(f"Hill climbing path was: Galway → Athlone → Belfast → Dublin → Waterford, cost = 705 km")
print(f"\nA* is not sidetracked because f(Athlone) = g + h = 80 + 90 = 170, while")
```

```
print(f"f(Limerick) = 105 + 170 = 275. Although Athlone's f is lower initially,")
print(f"the subsequent f-values through Athlone quickly exceed the Limerick path.")
```

```
A* with Athlone foothill:
===========================================================================================

Step 1: Visit Galway [f=200, g=0, h=200]
  CLOSED: ['Galway']

Step 2: Visit Athlone [f=170, g=80, h=90]
  OPEN:   Belfast(f=576, g=306, h=270), Limerick(f=275, g=105, h=170)
  CLOSED: ['Athlone', 'Galway']

Step 3: Visit Limerick [f=275, g=105, h=170]
  OPEN:   Belfast(f=576, g=306, h=270)
  CLOSED: ['Athlone', 'Galway', 'Limerick']

Step 4: Visit Cork [f=346, g=226, h=120]
  OPEN:   Belfast(f=576, g=306, h=270)
  CLOSED: ['Athlone', 'Cork', 'Galway', 'Limerick']

Step 5: Visit Waterford [f=352, g=352, h=0]
  OPEN:   Belfast(f=576, g=306, h=270), Dublin(f=576, g=446, h=130)
  CLOSED: ['Athlone', 'Cork', 'Galway', 'Limerick', 'Waterford']

A* path: Galway → Limerick → Cork → Waterford, cost = 352 km
Hill climbing path was: Galway → Athlone → Belfast → Dublin → Waterford, cost = 705 km

A* is not sidetracked because f(Athlone) = g + h = 80 + 90 = 170, while
f(Limerick) = 105 + 170 = 275. Although Athlone's f is lower initially,
the subsequent f-values through Athlone quickly exceed the Limerick path.
```

5.5 Verifying with NetworkX

In [ ]:
```
# NetworkX A* — pass our heuristic function
def nx_heuristic(n, goal):
    return heuristic[n]

nx_path = nx.astar_path(ireland, 'Galway', 'Waterford',
                        heuristic=nx_heuristic, weight='weight')
# TODO: Run the NetworkX function to compute the length of the shortest path
#       between source and target using A*


print(f"Our A* path:     {astar_path}, cost = {astar_cost} km")
print(f"NetworkX A* path: {list(nx_path)}, cost = {nx_cost} km")
print(f"Match: {astar_path == list(nx_path) and astar_cost == nx_cost}")
```

```
Our A* path:     ['Galway', 'Limerick', 'Cork', 'Waterford'], cost = 352 km
NetworkX A* path: ['Galway', 'Limerick', 'Cork', 'Waterford'], cost = 352 km
Match: True
```

Part 6: Algorithm Comparison

Let's compare all algorithms on the same problem.

In [ ]:
```
# Run BFS (uninformed search) for comparison (finds fewest-edge path)
# We compute its actual cost only for comparison with the other algorithms.
def bfs(graph, start, goal):
    visited = set()
    queue = deque([(start, [start], 0)])
    expanded = 0
    while queue:
        node, path, cost = queue.popleft()
        if node in visited:
            continue
        visited.add(node)
        expanded += 1
        if node == goal:
            return path, cost, expanded
        for neighbor in graph.neighbors(node):
            if neighbor not in visited:
                dist = graph[node][neighbor]['weight']
                queue.append((neighbor, path + [neighbor], cost + dist))
    return None, 0, expanded

bfs_path, bfs_cost, bfs_exp = bfs(ireland, 'Galway', 'Waterford')

print("Algorithm Comparison: Galway → Waterford")
print("=" * 75)
print(f"{'Algorithm':<25} {'Path':>35} {'Cost (km)':>10} {'Expanded':>9}")
print("-" * 75)

results = [
    ("BFS", bfs_path, bfs_cost, bfs_exp),
    ("Hill Climbing", hc_path, hc_cost, len(hc_trace)),
    ("Greedy Best-First", gbf_path, gbf_cost, gbf_exp),
    # TODO: Print Beam Search (w=2) results
```

```
    # TODO: Print our A* results

]

for name, path, cost, exp in results:
    path_str = ' → '.join(path) if path else 'NOT FOUND'
    print(f"{name:<25} {path_str:>35} {cost:>10} {exp:>9}")
```

```
Algorithm Comparison: Galway → Waterford
============================================================================
Algorithm                                                Path  Cost (km)  Expanded
----------------------------------------------------------------------------
BFS                        Galway → Limerick → Cork → Waterford        352         6
Hill Climbing              Galway → Limerick → Cork → Waterford        352         4
Greedy Best-First          Galway → Limerick → Cork → Waterford        352         4
Beam Search (w=2)          Galway → Belfast → Dublin → Waterford        631         6
A*                         Galway → Limerick → Cork → Waterford        352         4
```

Properties of Informed Search Algorithms

Algorithm	Complete?	Optimal?	Uses $h(n)$?	Uses $g(n)$?	Backtracks?
Hill Climbing	No	No	Yes	No	No
Greedy Best-First	Yes*	No	Yes	No	Yes
Beam Search	No	No	Yes	No	Limited
A*	Yes	Yes	Yes	Yes	Yes

* On finite graphs with cycle detection.

> **A* is optimal because it uses both $g(n)$ and $h(n)$.** Greedy best-first and hill climbing only use $h(n)$, so they ignore the cost already paid to reach a node. This is why they can find expensive paths — they don't know how far they've already travelled.

Part 7: A* with Different Evaluation Functions

7.1 Grid World with Terrain

So far, $g(n)$ has simply been the sum of edge distances. But A*'s evaluation function can incorporate any cost model. To demonstrate this, we'll use a grid world where different cells have different terrain costs*:

- **Flat land** (cost 1): Easy to traverse.
- **Hills** (cost 3): Harder, takes more effort.
- **Swamp** (cost 5): Very expensive to cross.
- **Walls** (cost 0): Impassable.

The same grid can be searched with two different cost models:

1. **Uniform cost ($g(n) =$ number of steps):** Finds the path with the **fewest steps** (shortest distance).
2. **Terrain cost ($g(n) =$ sum of terrain costs):** Finds the path with the **lowest total cost** (cheapest path).

This shows how the same algorithm (A*) produces different optimal paths depending on what you're optimising for.

7.2 Building the Grid World

In [ ]:
```
# Grid world: 7 rows x 10 columns
# 1 = flat, 3 = hills, 5 = swamp, 0 = wall
terrain = np.ones((7, 10), dtype=int)

# Wall across the middle (column 5), rows 1-5
for r in range(1, 6):
    terrain[r, 5] = 0

# Top gap: passable but EXPENSIVE (swamp)
terrain[0, 4] = 5
terrain[0, 5] = 5
terrain[0, 6] = 5
terrain[1, 4] = 3  # hills near the gap
terrain[1, 6] = 3

# Bottom gap: open and flat (but longer route)
terrain[6, 5] = 1  # flat

# Some hills on the left side
terrain[3, 3] = 3
terrain[3, 4] = 3
terrain[4, 4] = 3

start = (3, 0)   # middle-left
goal = (3, 9)    # middle-right

# Display the grid
terrain_names = {0: 'Wall', 1: 'Flat', 3: 'Hills', 5: 'Swamp'}
print("Terrain Grid (S = start, G = goal):")
for r in range(7):
```

```
    row_str = ""
    for c in range(10):
        if (r, c) == start:
            row_str += " S "
        elif (r, c) == goal:
            row_str += " G "
        elif terrain[r, c] == 0:
            row_str += " ■ "
        else:
            row_str += f" {terrain[r,c]} "
    print(row_str)
print("\nLegend: 1=Flat, 3=Hills, 5=Swamp, ■=Wall")
```

```
Terrain Grid (S = start, G = goal):
 1  1  1  1  5  5  5  1  1  1
 1  1  1  1  3  ■  3  1  1  1
 1  1  1  1  1  ■  1  1  1  1
 S  1  1  3  3  ■  1  1  1  G
 1  1  1  1  3  ■  1  1  1  1
 1  1  1  1  1  ■  1  1  1  1
 1  1  1  1  1  1  1  1  1  1

Legend: 1=Flat, 3=Hills, 5=Swamp, ■=Wall
```

7.3 A* with Different Cost Functions

In [ ]:

```
def manhattan_distance(a, b):
    """Manhattan distance heuristic."""
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def a_star_grid(grid, start, goal, use_terrain_cost=True):
    """
    A* on a grid world.

    Parameters:
        grid:              2D numpy array of terrain costs (0 = wall)
        start, goal:       (row, col) tuples
        use_terrain_cost:  If True, step cost = terrain value.
                           If False, step cost = 1 (uniform).

    Returns:
        path, cost, expanded
    """
    rows, cols = grid.shape
    visited = set()
    g_costs = {start: 0}
    # Priority queue: (f, g, node, path)
    h_start = manhattan_distance(start, goal)
    pq = [(h_start, 0, start, [start])]
    expanded = 0

    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        expanded += 1

        if node == goal:
            return path, g, expanded, visited

        r, c = node
        for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr, nc] != 0:
                neighbor = (nr, nc)
                step_cost = grid[nr, nc] if use_terrain_cost else 1
                new_g = g + step_cost
                if neighbor not in visited and (neighbor not in g_costs or new_g < g_costs[neighbor]):
                    g_costs[neighbor] = new_g
                    h = manhattan_distance(neighbor, goal)
                    # Heuristic scaled by minimum step cost for admissibility
                    new_f = new_g + h * (1 if use_terrain_cost else 1)
                    heapq.heappush(pq, (new_f, new_g, neighbor, path + [neighbor]))

    return None, 0, expanded, visited

# A* with uniform cost (shortest distance)
path_dist, cost_dist, exp_dist, visited_dist = a_star_grid(
    terrain, start, goal, use_terrain_cost=False)

# TODO: Run A* considering the terrain cost
# A* with terrain cost (cheapest path)


# Compute cross-metrics
dist_terrain_cost = sum(terrain[r, c] for r, c in path_dist[1:])
cheap_steps = len(path_cheap) - 1

print(f"Shortest-distance path (uniform cost):")
print(f"  Steps: {len(path_dist)-1}, Terrain cost: {dist_terrain_cost}, Expanded: {exp_dist}")
print(f"\nCheapest-terrain path:")
```

```
print(f"  Steps: {cheap_steps}, Terrain cost: {cost_cheap}, Expanded: {exp_cheap}")

if dist_terrain_cost != cost_cheap:
    print(f"\nThe shortest path costs {dist_terrain_cost} in terrain — "
          f"{dist_terrain_cost - cost_cheap} more than the cheapest path!")
    print(f"The cheapest path takes {cheap_steps - (len(path_dist)-1)} extra steps "
          f"but saves {dist_terrain_cost - cost_cheap} in terrain cost.")
```

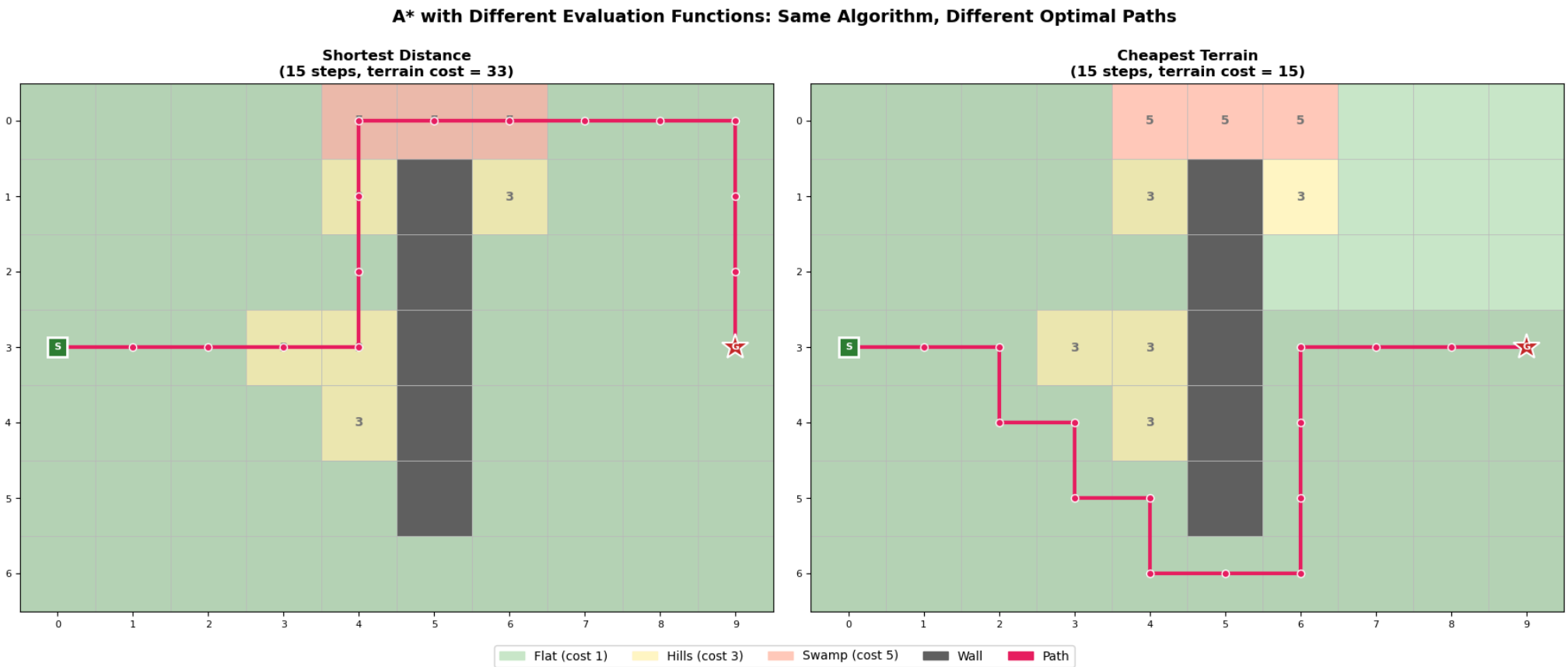
```
Shortest-distance path (uniform cost):
  Steps: 15, Terrain cost: 33, Expanded: 65

Cheapest-terrain path:
  Steps: 15, Terrain cost: 15, Expanded: 51

The shortest path costs 33 in terrain — 18 more than the cheapest path!
The cheapest path takes 0 extra steps but saves 18 in terrain cost.
```

7.4 Visualising Both Paths

In [ ]:

```
terrain_cmap = {0: '#616161', 1: '#C8E6C9', 3: '#FFF9C4', 5: '#FFCCBC'}
terrain_labels = {0: 'Wall', 1: 'Flat (1)', 3: 'Hills (3)', 5: 'Swamp (5)'}

fig, axes = plt.subplots(1, 2, figsize=(18, 7))

title_dist = f'Shortest Distance\n({len(path_dist)-1} steps, terrain cost = {dist_terrain_cost})'
title_cheap = f'Cheapest Terrain\n({cheap_steps} steps, terrain cost = {cost_cheap})'

paths_info = [
    (axes[0], path_dist, title_dist, visited_dist),
    (axes[1], path_cheap, title_cheap, visited_cheap),
]

for ax, path, title, visited_set in paths_info:
    rows, cols = terrain.shape

    # Draw terrain
    for r in range(rows):
        for c in range(cols):
            color = terrain_cmap[terrain[r, c]]
            # Slightly highlight visited cells
            if (r, c) in visited_set and terrain[r, c] != 0:
                import matplotlib.colors as mcolors
                rgb = mcolors.to_rgb(color)
                color = tuple(max(0, v - 0.07) for v in rgb)

            rect = plt.Rectangle((c - 0.5, r - 0.5), 1, 1,
                                 facecolor=color, edgecolor='#BBBBBB', linewidth=0.5)
            ax.add_patch(rect)

            # Show terrain cost in cell
            if terrain[r, c] > 1:
                ax.text(c, r, str(terrain[r, c]), ha='center', va='center',
                        fontsize=10, color='#777777', fontweight='bold')

    # Draw path
    if path:
        path_r = [p[0] for p in path]
        path_c = [p[1] for p in path]
        ax.plot(path_c, path_r, 'o-', color='#E91E63', linewidth=3,
                markersize=6, markeredgecolor='white', markeredgewidth=1, zorder=5)

    # Start and goal markers
    ax.plot(start[1], start[0], 's', color='#2E7D32', markersize=16, zorder=6,
            markeredgecolor='white', markeredgewidth=2)
    ax.plot(goal[1], goal[0], '*', color='#C62828', markersize=22, zorder=6,
            markeredgecolor='white', markeredgewidth=1.5)
    ax.text(start[1], start[0], 'S', ha='center', va='center', fontsize=8,
            color='white', fontweight='bold', zorder=7)
    ax.text(goal[1], goal[0], 'G', ha='center', va='center', fontsize=8,
            color='white', fontweight='bold', zorder=7)

    ax.set_xlim(-0.5, cols - 0.5)
    ax.set_ylim(rows - 0.5, -0.5)
    ax.set_aspect('equal')
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.set_xticks(range(cols))
    ax.set_yticks(range(rows))
    ax.tick_params(labelsize=8)

# Shared legend
legend_elements = [
    mpatches.Patch(color='#C8E6C9', label='Flat (cost 1)'),
    mpatches.Patch(color='#FFF9C4', label='Hills (cost 3)'),
    mpatches.Patch(color='#FFCCBC', label='Swamp (cost 5)'),
    mpatches.Patch(color='#616161', label='Wall'),
    mpatches.Patch(color='#E91E63', label='Path'),
]
fig.legend(handles=legend_elements, loc='lower center', ncol=5, fontsize=10,
           bbox_to_anchor=(0.5, -0.02))

plt.suptitle('A* with Different Evaluation Functions: Same Algorithm, Different Optimal Paths',
             fontsize=14, fontweight='bold', y=1.05)
```

```
plt.tight_layout()
plt.subplots_adjust(bottom=0.08)
plt.show()
```



7.5 The Evaluation Function Determines the Solution

The key insight is that A* finds the **optimal solution for whatever $f^*(n)$ you define**:

Cost Model	$g(n)$	$h(n)$	Optimises for
Uniform cost	Steps taken	Manhattan distance	**Shortest distance**
Terrain cost	Sum of terrain costs	Manhattan distance × min cost	**Cheapest path**

Both use the same algorithm. Both are optimal. They just optimise different objectives.

This principle extends to any problem where you can define meaningful costs on edges — travel time, fuel consumption, danger level, etc.

Summary

In this lab, we:

1. **Built** a weighted graph (Ireland map) with heuristic values using NetworkX.
2. **Verified** heuristic admissibility and consistency (the triangle inequality).
3. **Implemented** hill climbing, greedy best-first search, beam search, and A* **from scratch**.
4. **Demonstrated** hill climbing's vulnerability to foothills using the Athlone example.
5. **Traced** A*'s OPEN and CLOSED lists step by step — the format used in exam questions.
6. **Compared** all algorithms on the same problem.
7. **Showed** how A* adapts to different cost models using a grid world with terrain costs.

Key takeaways:

- Hill climbing is fast but unreliable — local search can't see beyond the current neighbourhood.
- Greedy best-first is better (uses a global queue) but still ignores path cost.
- **A* combines $g(n)$ and $h(n)$ to guarantee optimal solutions**, given an admissible heuristic.
- The evaluation function $f(n) = g(n) + h(n)$ is the key — **what you put into $g(n)$ determines what A* optimises for**.
- A*'s main weakness is space complexity $O(b^d)$. Algorithms like IDA* and RBFS address this (see the self-study notebook).

Practice exercises:

- Try to hand-draw a semantic graph to represent this problem.
- Try to hand-trace the open and closed lists for at least the first 5 iterations of the cheapest terrain search.